

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/36409243>

Proposta de Utilização do Diagrama de Seqüência para Definição de Casos de Teste de Unidade

Article · December 2008

DOI: 10.22456/2175-2745.7029 - Source: OAI

CITATIONS

0

READS

548

5 authors, including:



Fabiane Barreto Vavassori Benitti

Federal University of Santa Catarina

49 PUBLICATIONS 828 CITATIONS

SEE PROFILE

Some of the authors of this publication are also working on these related projects:



SESRA - Software Engineering Systematic Literature App [View project](#)



SE-RPG [View project](#)

Proposta de Utilização do Diagrama de Sequência para Definição de Casos de Teste de Unidade

Antonio Carlos Silva¹

Fabiane Barreto Vavassori Benitti¹

Resumo: A etapa de teste de software tem se mostrado cada vez mais relevante no processo de desenvolvimento de software, impactando diretamente na qualidade do produto. Dentre os testes aplicados no software, observa-se que o teste de unidade não apresenta um artefato alinhado com o padrão adotado para especificação de software (UML). Neste sentido, apresenta-se uma proposta de utilização do diagrama de sequência como artefato para definição de casos de teste, especificamente voltado a teste de unidade, tornando o planejamento de testes independente da linguagem de programação, ampliando as possibilidades de utilização do artefato. A adequação do diagrama proposto é demonstrada através de uma ferramenta que permite a geração do código para execução com *framework* JUnit.

Palavras-chave: Engenharia de software, teste de software, teste de unidade.

Abstract: The step of software testing has been increasingly relevant in the software development process, directly impacting on the quality of the product. Among the tests applied in the software, it appears that the unit test does not present an artifact aligned with the standard adopted for specification of software (UML). In this sense, it presents a proposal for using the sequence diagram of how to define artifact of test cases, specifically geared to unit test, making the planning of tests regardless of programming language, expanding the possibilities for using the artifact. The suitability of the proposed diagram is demonstrated by using a tool that allows the generation of code to run on JUnit framework.

Keyword: Software engineering, software testing, unit test.

1 Introdução

O teste de software é uma das etapas no ciclo de desenvolvimento de software que tem como principal objetivo revelar a presença de defeitos em um software. Esta etapa é

¹ Centro de Educação de Ciências Tecnológicas da Terra e do Mar, UNIVALI, Itajaí, SC - Brasil
{antonio.carlos, fabiane.benitti @univali.br}

compreendida por sistemáticas aplicações de testes ao longo de todo o processo de desenvolvimento [1]. A etapa de testes é definida por Hetzel [2] como: “o processo de executar um programa ou sistema com a finalidade de encontrar erros. Teste é a medida de qualidade do software”.

Um dos testes iniciais aplicados a um software é o teste de unidade, que acontece antes mesmo da total codificação do sistema. O teste de unidade, conhecido também por teste de componente ou teste de módulo, é definido por Thomas *et al* [3], como um teste projetado para verificar uma única unidade de software de forma isolada das demais unidades. Em projetos orientados a objetos, uma unidade de implementação é uma classe ou um método de uma classe.

A técnica de desenvolvimento TDD (*Test Driven Development*), proposta por Beck [4], apóia-se na aplicação de testes de unidade antes da codificação das unidades propriamente ditas. Alguns dos benefícios atribuídos ao uso da técnica referem-se ao fato de que toda unidade de software possuirá um conjunto de testes que verifique seu funcionamento, baseado nos requisitos estabelecidos para esta unidade, assim antecipando a detecção de erros presentes na codificação. A etapa inicial envolvida na aplicação da técnica é o planejamento dos casos de teste, momento em que são definidos os detalhes envolvidos na execução dos testes, como os dados de entrada e os resultados esperados.

Para o teste de unidade a etapa de planejamento é comumente confundida com a codificação dos *drivers* e *stubs*, resultando em pouca ou nenhuma documentação aos casos de teste, restando apenas o próprio código fonte dos testes como especificação [4]. Esta deficiência na documentação dos casos de teste, identificada também por Runeson [5], pode ser explicada pela falta de uma linguagem padrão para definição dos casos de teste e uma ferramenta que apóie este processo [6]. Neste ponto, Biasi e Becker [6] apresentam alguns problemas na utilização de ferramentas para automatização de testes: (i) o esforço necessário para preparar o código auxiliar; (ii) nem sempre há distinção clara entre as atividades de especificação dos casos de teste e de programação do código auxiliar; e (iii) dependência do caso de teste da linguagem de programação.

Desta forma, este artigo propõe a utilização do diagrama de sequência, definido na UML (*Unified Modeling Language*), como artefato para definição de casos de teste (detalhado na seção 3). Também apresenta, na seção 4, uma ferramenta que visa auxiliar na definição dos casos de teste, bem como automatizar a geração do código fonte de execução dos testes utilizando o *framework* JUnit. Na seção 2 são apresentados alguns dos pontos fundamentais da técnica TDD no que se refere a testes de unidade. Por fim, as considerações finais e direcionamentos para trabalhos futuros são contempladas na seção 5.

2 Test Driven Development - TDD

O TDD (*Test Driven Development*) ou *Test First* tem como base a definição e aplicação de testes de unidade antes mesmo da codificação das unidades envolvidas.

Destaca-se ainda que um código para uma unidade de software só pode ser desenvolvido se houver um conjunto de casos de teste para verificá-la [7].

A técnica de desenvolvimento TDD consiste em três etapas (Figura 1): (i) definição dos casos de teste; (ii) codificação das classes com o objetivo de atender aos casos de teste; e (iii) refatoração. Cada uma destas etapas é aplicada a cada unidade de software [4].

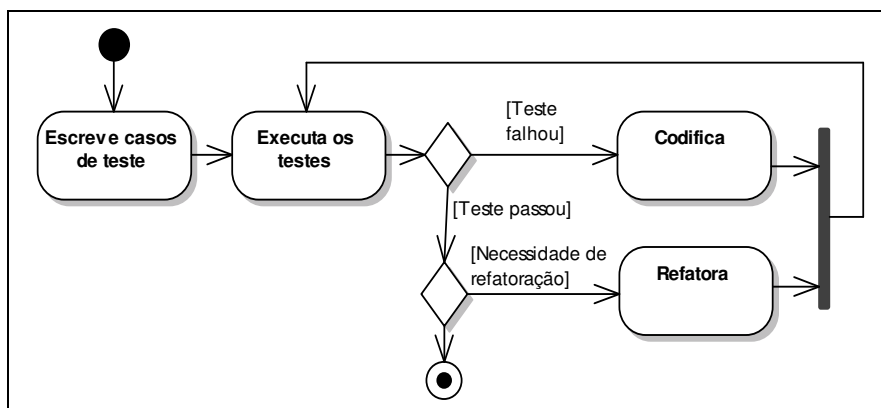


Figura 1. Diagrama de atividades do TDD

A definição dos casos de teste ganha destaque se comparado a abordagem tradicional de teste de software, uma vez que este guiará a codificação do sistema. Nesta etapa são identificados os *drivers* e *stubs* para posterior codificação [7].

São os *drivers* que determinam a execução dos casos de teste definindo as mensagens entre os objetos envolvidos e as verificações estabelecidas a partir dos resultados esperados [8]. Comumente este código é desenvolvido para atender a uma ferramenta que automatize a execução dos testes², como por exemplo, o JUnit da família xUnit [9], responsável pela automatização de testes desenvolvidos em linguagem Java.

Os *stubs* de testes têm por objetivo substituir outras unidades de software envolvidas em um caso de teste, eliminando a dependência entre as unidades [8]. O uso de *stubs* permite atender ao princípio de isolamento das unidades, em que cada unidade deve ser testada de forma isolada das demais [3] e [10].

A codificação das classes é efetuada com base nos casos de teste estabelecidos, tendo como objetivo o êxito na execução dos testes. Na etapa de refatoração o código desenvolvido para atender aos casos de testes é avaliado de forma a verificar questões como flexibilidade, reaproveitamento de código, desempenho, clareza na solução e duplicidade no código [7].

² A automatização dos testes de unidade é um dos requisitos no uso da técnica TDD, sabendo-se que os testes são executados com grande frequência na implementação das unidades [4].

3 Definindo casos de teste

Na etapa de planejamento dos casos de teste são especificados os dados de entrada e os resultados esperados, bem como todo o fluxo de execução do caso de teste como, por exemplo, as mensagens trocadas entre os objetos.

A pouca padronização na definição de casos de teste é destacada por Biasi e Becker [6], Badri, Badri e Bourque-Fortin [11] e Linzhang et al. [12]. Observa-se, ainda, que no padrão proposto pela IEEE [10] para testes de unidade também não é abordada uma linguagem para definição de casos de testes. Dentre as soluções já propostas, destaca-se a de Xu, Xu e Nygard (2005 *apud* [11]), que propõem a definição dos casos de teste tendo como base o diagrama de estados. No entanto, uma das limitações desta proposta refere-se ao fato de que os elementos deste diagrama não estão diretamente relacionados às unidades de software. A fim de contornar este problema Badri, Badri e Bourque-Fortin [11] apresentam novos conceitos e definições ao diagrama de estados. Assim, a proposta de uma representação visual, baseado na UML e utilizando tão somente os recursos já especificados na linguagem, favorece a aplicabilidade por parte dos analistas e o suporte por ferramentas já existentes.

O diagrama de seqüência é um dos diagramas disponibilizados pela UML para modelagem das interações na etapa de projeto, no entanto, propõem-se a sua utilização para etapa de teste. A Figura 2 resume a proposta de utilização do diagrama de seqüência como linguagem visual para definição de casos de teste, destacando os principais elementos envolvidos.

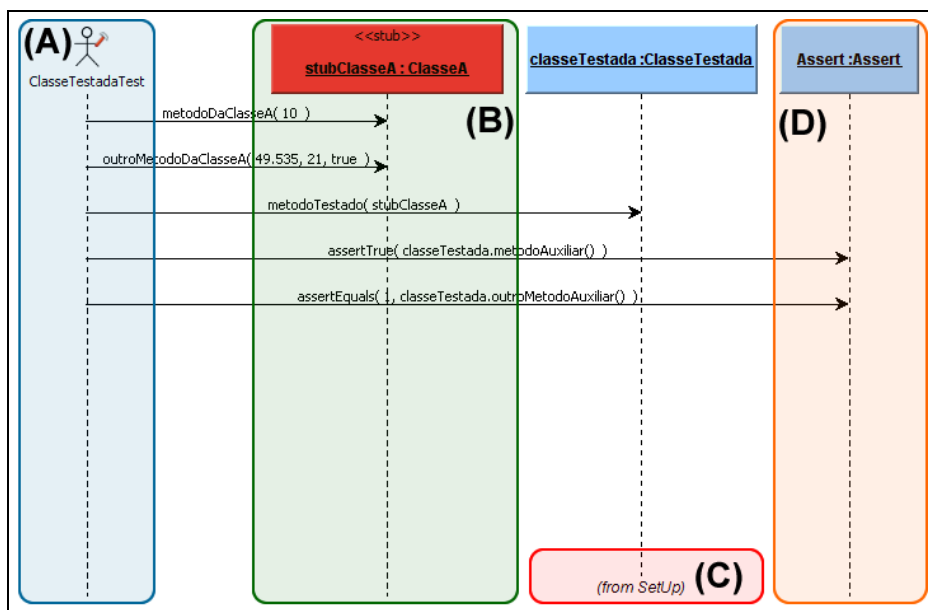


Figura 2. Diagrama de sequência utilizado para definição dos casos de teste

Durante a elaboração do caso de teste devem ser definidas mensagens entre o *driver* (gerenciador dos casos de teste) e os objetos envolvidos neste. Para representação do *driver* utilizou-se a figura do ator, elemento já existente no diagrama de sequência, denotando o responsável por invocar os métodos dos objetos envolvidos nos casos de teste, sendo apresentado à esquerda do diagrama, como demonstrado na Figura 2 (A).

Conforme abordado na seção 2, a utilização de *stubs* em casos de teste é fundamental para garantir o isolamento da unidade testada, desta forma, um *stub* no diagrama proposto é representado como um objeto acrescido do estereótipo <<stub>>, conforme a Figura 2 (B).

Outro recurso pertinente quando da criação de casos de teste refere-se ao *SetUp*. Este recurso permite definir objetos compartilhados por todos os casos de teste de uma classe, ou seja, considerados como de uso global. Assim, quando utilizado o *SetUp* de uma classe no diagrama de sequência é exibida ao fim da linha de vida de cada objeto a mensagem *from SetUp*, conforme a Figura 2 (C), evidenciando o uso de objetos compartilhados.

Para todo caso de teste definido deve ser incluído um objeto *Assert*, como demonstrado na Figura 2 (D). O elemento *Assert* representa a classe de mesmo nome do *framework* xUnit sendo responsável pela verificação do caso de teste. Ressalta-se que esta classe é padronizada nos *frameworks* da família xUnit, ou seja, a sua utilização independe da linguagem de implementação.

Destaca-se que para definição de um caso de teste utilizando o diagrama de sequência basta que sejam definidos os objetos envolvidos no teste e as mensagens trocadas entre estes, com isso obtêm-se um padrão baseado em uma linguagem já estabelecida e de conhecimento de muitos dos analistas e testadores envolvidos no processo de desenvolvimento de software.

Com o objetivo de demonstrar a viabilidade na utilização deste diagrama como forma de especificação de casos de testes, a seção 4 apresenta a proposta de uma ferramenta que permite a geração automática do código de teste de unidade a partir de um caso de teste, definido visualmente através do diagrama de sequência, elaborado conforme descrito nesta seção.

4 Ferramenta para apoio no planejamento dos casos de teste

A ferramenta tem como objetivo permitir a criação de um projeto de casos de teste, focando testes de unidade considerando o contexto de um método (unidade). Para tanto, faz-se necessário que o modelo de classes do projeto testado esteja definido, pois este compreende a base dos testes. Sendo assim, a ferramenta apóia-se nas etapas definidas no diagrama de atividades ilustrado na Figura 3.

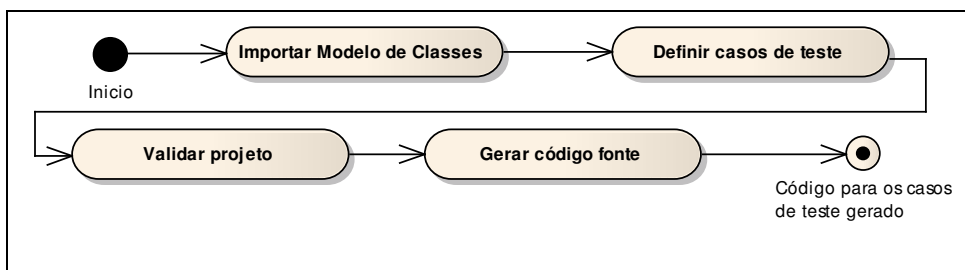


Figura 3. Diagrama de atividades para a ferramenta

1. Importar modelo de classes: permite conhecer o modelo de classes a ser testado, devendo ser compatível com o padrão XMI 2.1³;

2. Definir casos de teste: nesta etapa, utilizando o formato do diagrama de sequência proposto, o usuário define os casos de teste para os métodos do modelo importado;

3. Validar projeto: após a definição dos casos de teste a ferramenta permite a validação dos casos de teste, fazendo críticas a possíveis problemas encontrados nos casos de teste; e

³ Observa-se que as ferramentas efetuam implementações particulares do padrão XMI. Desta forma, a ferramenta apresentada neste artigo é compatível com a implementação das ferramentas Enterprise Architect 6.5 e 7.0 e Visual Paradigm for UML 6.0.

4. Gerar código fonte: por fim o usuário tem a opção de gerar o código fonte para os casos de teste do modelo, possibilitando sua execução pelo *framework* JUnit (inicialmente, para demonstrar a viabilidade do diagrama proposto, optou-se por testar sistemas desenvolvidos na linguagem Java).

Tendo em vista as etapas definidas, a Figura 4 apresenta o diagrama de casos de uso, representando as funcionalidades da ferramenta proposta.

Neste modelo de casos de uso percebe-se comportamentos básicos para operacionalizar a ferramenta, como abrir, salvar e fechar projeto (UC02, UC03 e UC04). Os demais casos de uso são o foco da ferramenta e, portanto, são detalhados na seqüência abordando sua funcionalidade juntamente com um estudo de caso.

A elaboração do caso de teste (UC05), através do diagrama de seqüência, parte da existência de um modelo de classes com seus respectivos métodos definidos. Sendo assim, considerando o modelo de classes ilustrado na Figura 5, é possível importá-lo na ferramenta (UC01) e iniciar a definição do caso de teste, através da interface ilustrada na Figura 6.

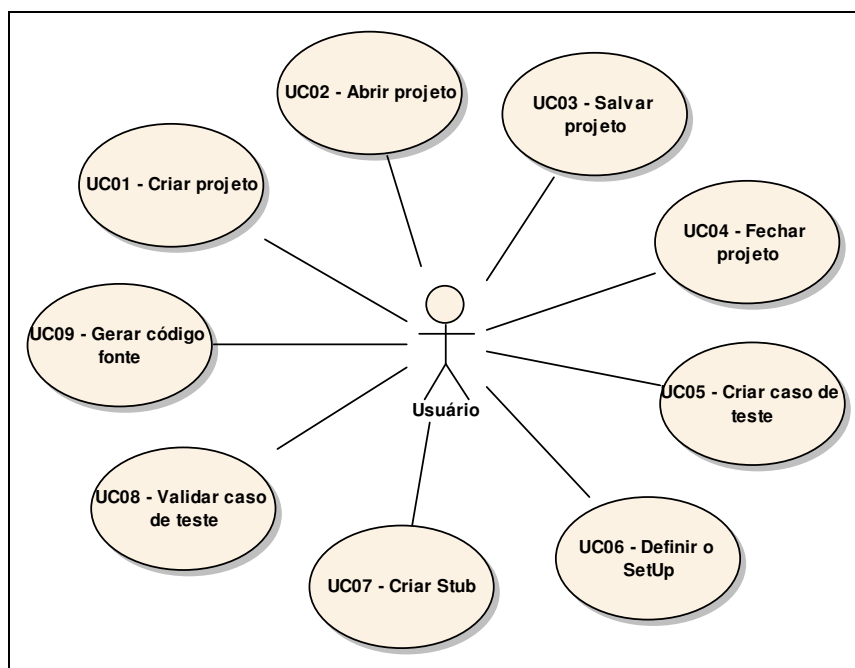


Figura 4. Casos de uso para a ferramenta proposta

O estudo de caso o qual é apresentado ao longo desta seção, a fim de demonstrar o funcionamento da ferramenta, possui como base o modelo de classes ilustrado pela Figura 5, onde o método *adicionaProduto* da classe *CarrinhoDeCompras* terá um caso de teste definido. Este caso de teste visa verificar se ao adicionar produtos ao *CarrinhoDeCompras* este responderá posteriormente de forma correta o número de produtos presentes no carrinho.

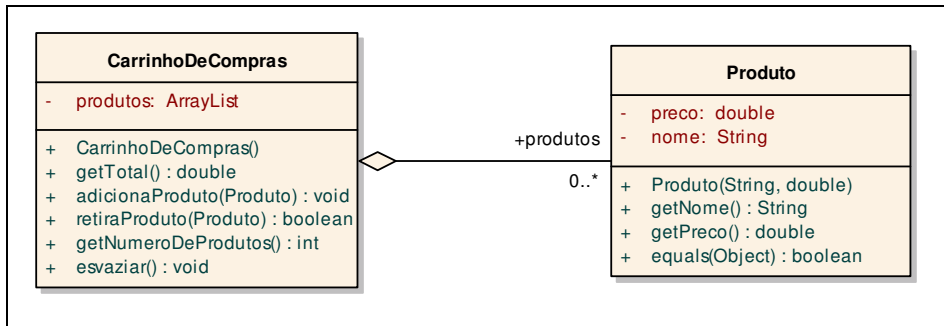


Figura 5. Diagrama de classes do estudo de caso

Alguns aspectos são importantes de serem destacados. Ao solicitar a criação de um novo caso de teste, a partir de um método presente no modelo e importado pela ferramenta (Figura 6 A), é apresentado ao usuário o diagrama de sequência com alguns elementos pré-definidos, conforme observado em (Figura 6 B):

- ✓ Gerenciador (*CarrinhoDeComprasTest*): responsável por gerenciar o caso de teste e do qual partirão todas as mensagens;
- ✓ *Stubs*: a ferramenta identifica com base na assinatura do método à ser testado, quais *Stubs* são necessários para sua chamada. Caso não exista nenhum *Stub* já criado do tipo necessário, um novo *Stub* é adicionado, devendo usuário definir seu comportamento; e
- ✓ Objeto Assert, que compõe o *framework* JUnit e é responsável pela verificação dos casos de teste.

No caso da figura 6, tem-se os elementos propostos para o caso de teste do método *AdicionaProduto* da classe *CarrinhoDeCompras*, apresentado na Figura 4. O *Stub* de nome *stubProduto*, presente no caso de teste, tem por objetivo permitir que o método *adicionaProduto* da classe *CarrinhoDeCompras* possa ser invocado sem a necessidade de que seja instanciado um objeto do tipo *Produto*, fortalecendo assim o princípio do isolamento destacado por IEEE [10].

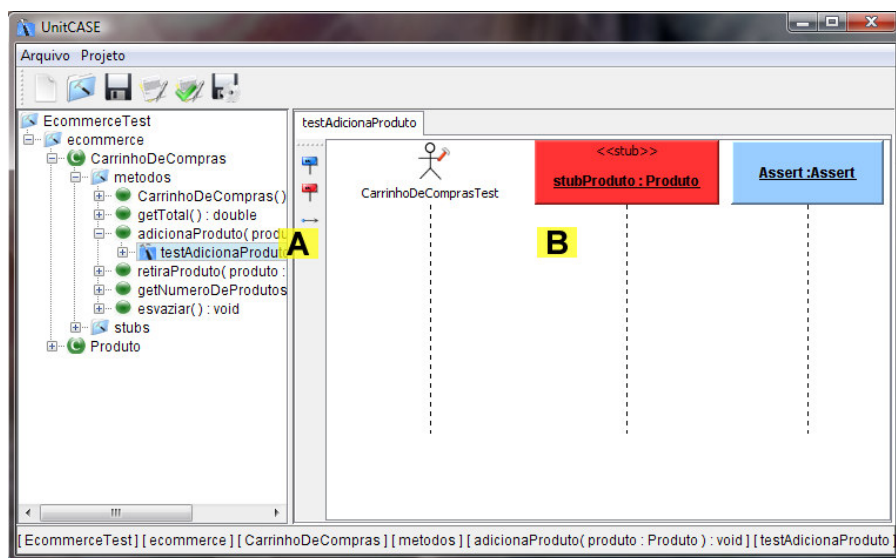


Figura 6. Interface para definição e visualização do caso de teste

O UC06 permite ao usuário a definição do *SetUp*, conforme exemplo ilustrado na Figura 7, em que é criado um objeto com nome de *carrinho*, como instância da classe *CarrinhoDeCompras*, sendo que para sua criação é selecionado o construtor padrão da classe.

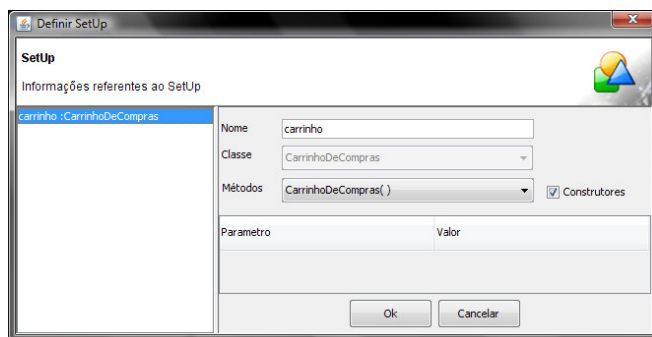


Figura 7. Interface para definição dos objetos do *SetUp*

Outra funcionalidade necessária é a criação de um *Stub* para uma classe (UC07). Esta funcionalidade é obtida a partir da seleção de uma das classes do modelo, quando a ferramenta apresenta a interface para criar um novo *Stub*, conforme demonstra a Figura 8a e incluí-lo em um caso de teste Figura 7b.

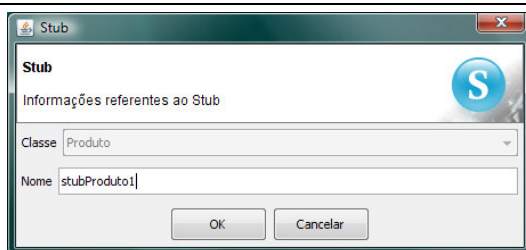


Figura 8a. Interface para criação de um novo *Stub*

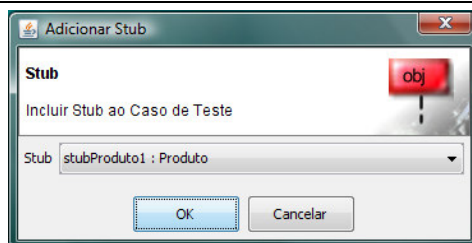


Figura 8b. Interface para adicionar um *Stub* ao caso de teste

Com os elementos do caso de teste definidos, pode-se então estabelecer as mensagens entre o gerenciador do caso de teste e os demais objetos (comportamento previsto para o UC05). Para definição de uma mensagem em um caso de teste é utilizada a interface apresentada na Figura 9, devendo ser indicado o método que será invocado e o valor dos parâmetros (se necessário).

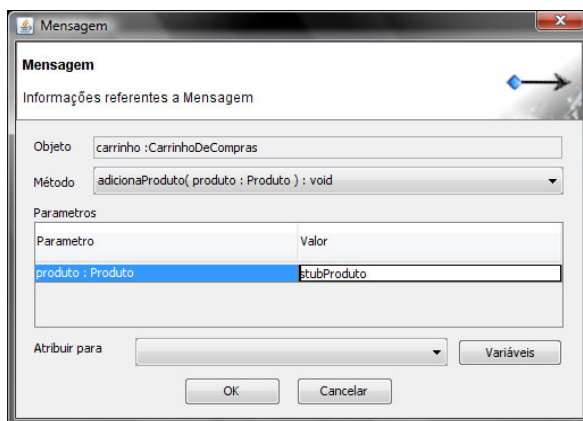


Figura 9. Interface para definição da mensagem

Para o caso de teste *testAdicionaProduto* foram definidas quatro mensagens, apresentadas na Figura 10. A primeira delas adiciona o *Stub stubProduto* ao objeto *carrinho* (Figura 10 A), a mensagem seguinte invoca o construtor da classe *Produto* (Figura 10 B), a fim de construir o objeto *produto1* o qual é adicionado ao objeto *carrinho* na mensagem seguinte (Figura 10 C). Por fim, a última mensagem do caso de teste verifica se o número de produtos do objeto *carrinho* corresponde ao valor esperado (Figura 10 D).

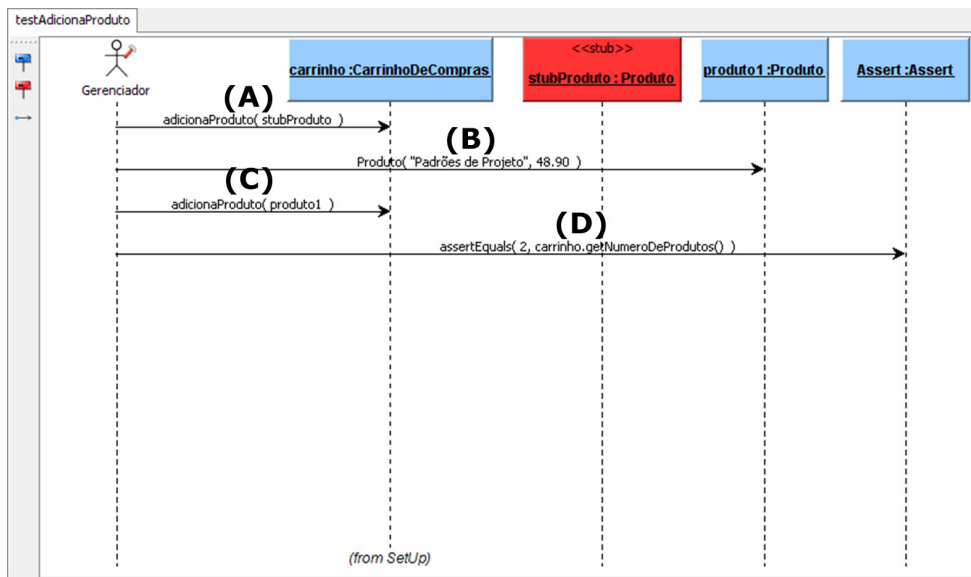


Figura 10. Caso de teste *testAdicionaProduto*

Utilizando a opção de validação (UC08), o sistema apresenta “críticas” quanto aos possíveis problemas encontrados nos casos de teste definidos pelo usuário. Estas críticas referem-se, por exemplo, a casos de teste que não possuem chamadas aos métodos da classe *Assert* do *framework* JUnit, ou métodos que não possuem casos de teste, ou mesmo casos de teste que não possuam pelo menos uma chamada ao método testado, conforme apresentado na Figura 11.

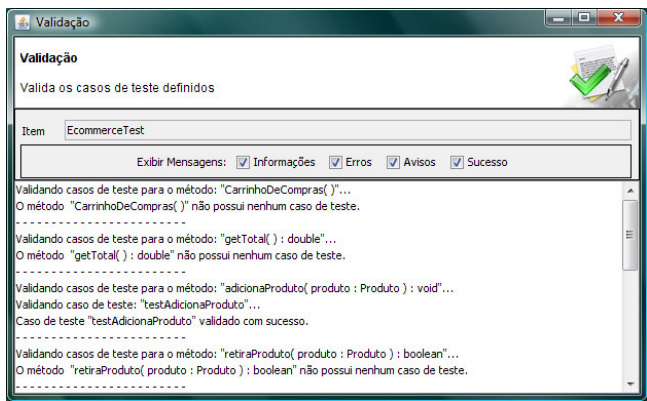


Figura 11. Interface para validação dos casos de teste

A geração do código fonte dos casos de teste definidos pelo usuário (UC09) é realizada a partir da interface demonstrada na Figura 12a. No exemplo apresentado, o código gerado pela ferramenta para o caso de teste *AdicionaProduto* pode ser visualizado na Figura 12b.

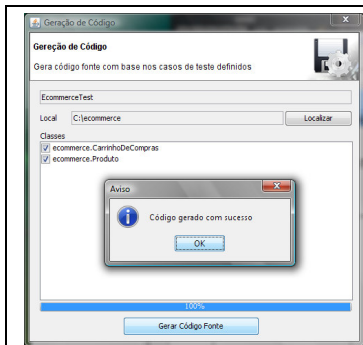


Figura 12a. Interface para geração de código fonte

```
package EcommerceTest.ecommerce;
import ...
public class CarrinhoDeComprasTest {
    ecommerce.CarrinhoDeCompras carrinho;
    @Before
    public void setUp() throws Exception {
        carrinho = new
ecommerce.CarrinhoDeCompras( );
    }
    @Test
    public void testAdicionaProduto() {
        ecommerce.Produto stubProduto =
EasyMock.createMock(ecommerce.Produto.class
);
        this.carrinho.adicionaProduto(
stubProduto );
        Produto produto1 = new Produto("Padrões
de Projeto", 48.90);
        this.carrinho.adicionaProduto( produto1
);
        Assert.assertEquals( 2,
carrinho.getNumeroDeProdutos() );
    }
}
```

Figura 12b. Código fonte gerado pela ferramenta

Com o objetivo de apresentar a estrutura utilizada para implementação da ferramenta são exibidos os diagramas de classes de domínio, sendo que cada diagrama representa um dos pacotes da ferramenta: *modelo* e *casoDeTeste*.

O diagrama de classes do pacote *modelo*, apresentado na Figura 13, tem por objetivo manter as informações necessárias sobre o modelo de classes do projeto ao qual estão sendo criados os casos de teste.

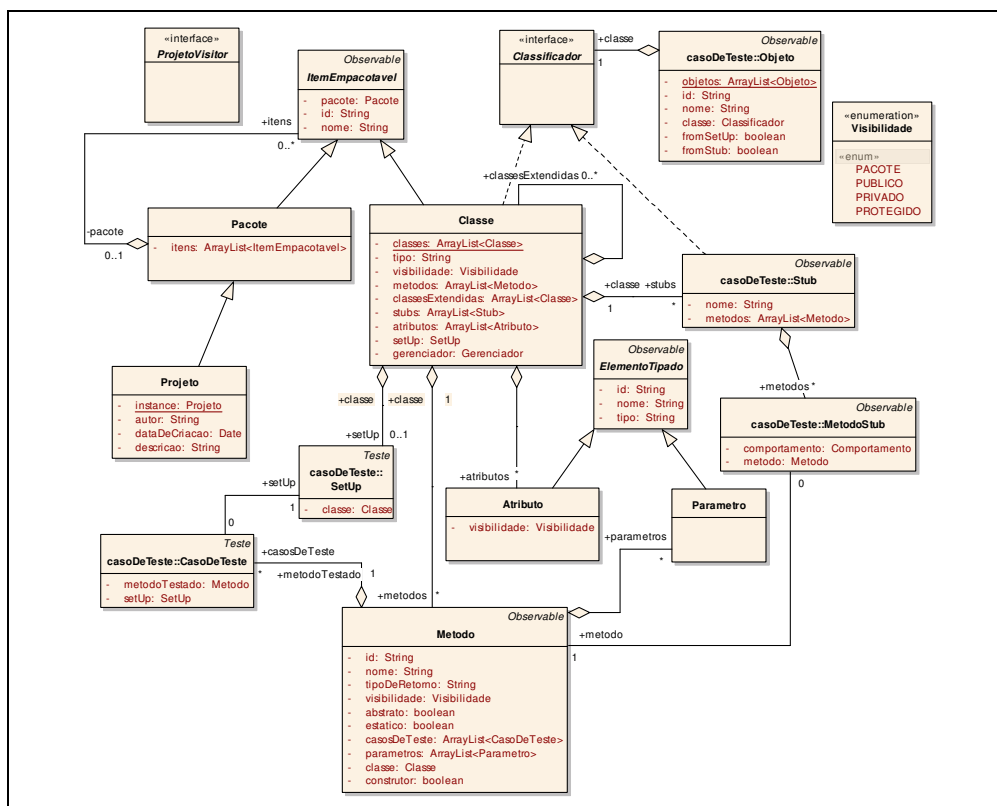
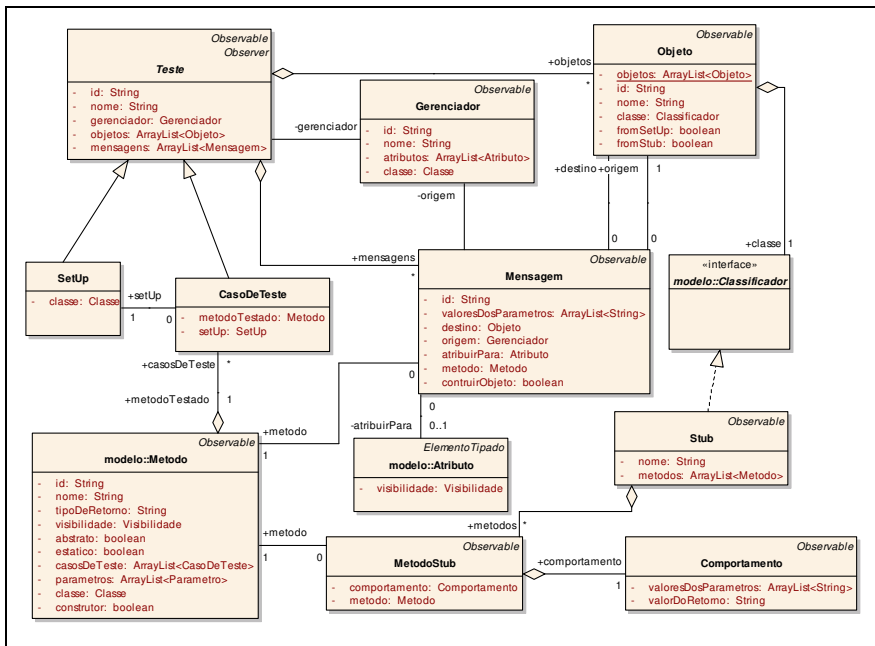


Figura 13. Diagrama de classes para o pacote *modelo* da ferramenta

Já o diagrama apresentado na Figura 14, contém as classes do pacote *casoDeTeste*, o qual é responsável por manter as informações sobre os casos de teste criados pelo usuário, o qual é criado com base na instância de objetos para as classes do projeto, permitindo então a troca de mensagens entre estes objetos.

Figura 14. Diagrama de classe para o pacote *casoDeTeste* para a ferramenta

5 Considerações finais

A principal contribuição deste artigo é apresentar uma proposta para utilização do diagrama de seqüência como artefato de definição de casos de teste visando auxiliar, por exemplo, a aplicação da técnica TDD.

O uso do diagrama de seqüência na definição dos casos de teste, ganha destaque à medida que este não foi concebido para este fim. Apesar disso, sua adequação se deu de forma natural, não exigindo a criação de qualquer elemento ou comportamento em especial, apenas utilizando de recursos já existentes na UML, como o estereótipo, utilizado para identificação do *stub*.

Observa-se ainda que o diagrama de seqüência como proposta para definição de casos de teste juntamente com a ferramenta desenvolvida se configuram como solução para os problemas citados em [6], dado que o código auxiliar é gerado automaticamente pela ferramenta. Os casos de testes podem ser definidos de forma visual sem qualquer contato com a linguagem de implementação, tendo como base apenas o modelo de classes do projeto.

O uso do diagrama de seqüência para definição dos casos de teste se demonstrou compatível com os recursos necessários para tal aplicação, dado que a ferramenta desenvolvida com base nesta definição pode gerar o código fonte para os casos de teste, tendo como base tão somente o diagrama de seqüência definido pelo usuário na própria ferramenta.

Por fim, acredita-se que o uso de uma linguagem visual para definição de casos de teste – linguagem esta que já é de domínio de parte dos analistas - permite uma melhor comunicação e interação entre os envolvidos no processo de desenvolvimento.

Referências

- [1] Macgregor, J.; Sykes, D. (2001). A Practical guide to testing object-oriented software. Addison-Wesley.
- [2] Hetzel, W. (1987). Guia completo do teste de software. Rio de Janeiro: Campus.
- [3] Thomas, J.; Young, M.; Brown, K.; Glover, A. (2004). Java testing patterns. Indianápolis: John Wiley.
- [4] Beck, K. (2002). Test driven development: by example. Boston: Addison-Wesley.
- [5] RUNESON, P. (2006). A survey of unit testing practices. In *IEEE Software*, v 23, Issue 4, p. 22 – 29, July-Aug.
- [6] Biasi, L.; Becker, K. (2006). Geração automatizada de drivers e stubs de teste para JUnit a partir de especificações U2TP. In *Proceedings of Brazilian Symposium of Software Engineering (SBES)*, Florianópolis – Santa Catarina.
- [7] Astels, D. (2003). Test Driven development: a practical guide. Prentice Hall.
- [8] Maldonado, J. C.; Fabbri, S. C. P. F.. (2001). Teste de software. In: Rocha, A. R. C. da; Maldonado, J. C.; Weber, K. C. (Coord.). Qualidade de software: teoria e prática. São Paulo: Prentice Hall, p. 73-84.
- [9] JUNIT (2007). JUnit FAQ. Disponível em: <http://junit.sourceforge.net/doc/faq/faq.htm>. Acesso em: 25 fev. 2007.
- [10] IEEE (1986). IEEE standard for software unit testing.
- [11] Badri, M.; Badri, L.; Bourque-Fortin, M.. (2005) Generating unit test sequences for aspect-oriented programs: towards a formal approach using UML state diagrams. In *3rd International Conference on Information and Communications Technology*. p. 237 – 253. Coimbatore, India.
- [12] Linzhang, W.; Jiesong Y.; Xiaofeng Y.; Jun H.; Xuandong L.; Guoliang Z. (2004). Generating test cases from UML activity diagram based on Gray-box method. In *11th Software Engineering Conference*. p. 284 – 291. Asia-Pacific.